21/10/2026

Experiment: 5

NAME: Gayathri. V

REG NO: 1911260168

SLOT: A

SEMESTER-MS2 September 2026

COURSE NAME: CSA4701 Deep Learning

5. Bias–Variance, Overfitting and Underfitting Analysis

Scenario

A student-performance prediction model uses a linear regression algorithm on nonlinear data. Compare training and testing errors to identify whether the model is underfitting**.**

Algorithm

1. Start the program.
2. Import NumPy, Pandas, Matplotlib, and Scikit-learn libraries.
3. Generate nonlinear student-performance data using study hours and marks.
4. Divide the dataset into training data and testing data.
5. Create a Linear Regression model.
6. Train the model using the training dataset.
7. Predict student marks for both training and testing datasets.
8. Calculate Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared values.
9. Compare the training and testing errors to identify underfitting.
10. Create a summary table displaying the calculated performance metrics.
11. Plot the actual student data and the linear regression line.
12. Plot a bar chart comparing training and testing errors.
13. Display the results and conclusion.
14. Stop the program.

Source Code

```
import numpy as np

import pandas as pd
```

```
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Create nonlinear student-performance data
np.random.seed(42)

study_hours = np.linspace(1, 10, 100)

# Nonlinear relationship between study hours and marks
marks = 20 + 7 * study_hours + 1.5 * study_hours**2
marks = marks + np.random.normal(0, 8, 100)

X = study_hours.reshape(-1, 1)
y = marks

# 2. Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train a simple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Predict training and testing marks
y_train_pred = model.predict(X_train)
```

```
y_test_pred = model.predict(X_test)

# 5. Calculate errors and R-squared scores
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

train_rmse = np.sqrt(train_mse)
test_rmse = np.sqrt(test_mse)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

# 6. Identify whether the model is underfitting
if train_mse > 100 and test_mse > 100:
    conclusion = "Underfitting: both errors are high"
elif test_mse > train_mse * 1.5:
    conclusion = "Possible overfitting: test error is higher"
else:
    conclusion = "No strong underfitting/overfitting signal"

# 7. Create summary table
summary = pd.DataFrame({
    "Metric": [
        "Training MSE",
        "Testing MSE",
        "Training RMSE",
        "Testing RMSE",
        "Training R-squared",
        "Testing R-squared"
```

```
    ],
    "Value": [
        train_mse,
        test_mse,
        train_rmse,
        test_rmse,
        train_r2,
        test_r2
    ]
})

print("STUDENT PERFORMANCE MODEL - SUMMARY TABLE")
print(summary.round(3).to_string(index=False))

print("\nConclusion:", conclusion)

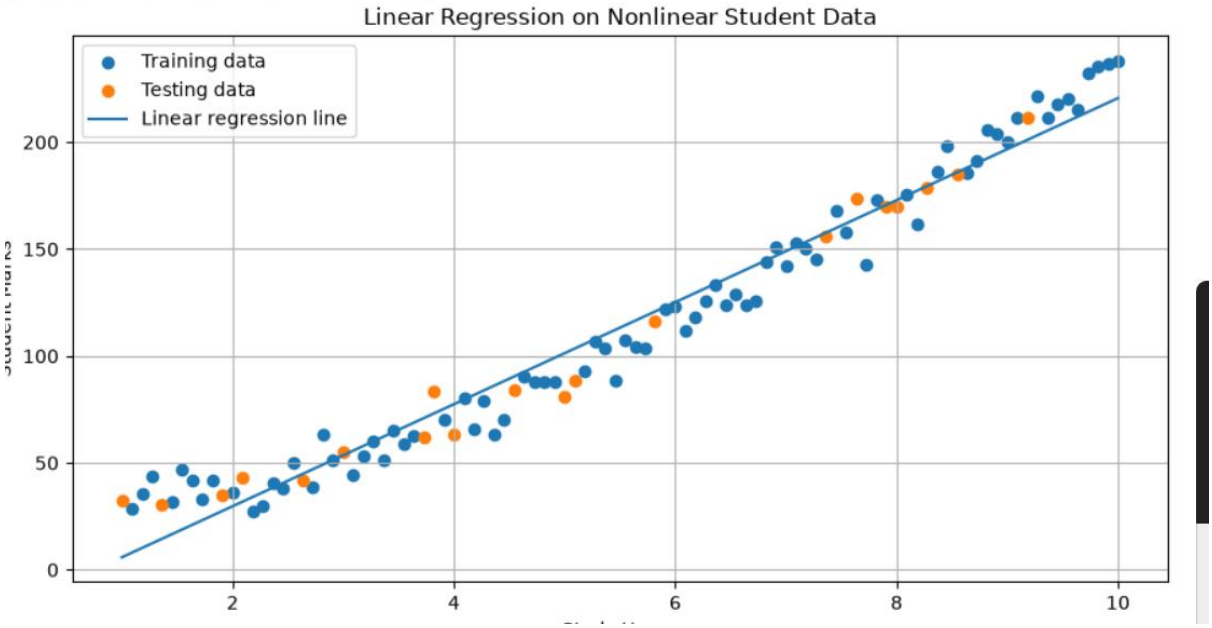
# 8. Plot actual data and linear regression line
plt.figure(figsize=(9, 5))
plt.scatter(X_train, y_train, label="Training data")
plt.scatter(X_test, y_test, label="Testing data")
x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
plt.plot(x_line, model.predict(x_line), label="Linear regression line")
plt.xlabel("Study Hours")
plt.ylabel("Student Marks")
plt.title("Linear Regression on Nonlinear Student Data")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# 9. Plot training and testing errors
plt.figure(figsize=(7, 5))
plt.bar(
    ["Training Error", "Testing Error"],
    [train_mse, test_mse]
)
plt.ylabel("Mean Squared Error")
plt.title("Comparison of Training and Testing Errors")
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```

Output

```
STUDENT PERFORMANCE MODEL - SUMMARY TABLE
             Metric    Value
      Training MSE  152.113
       Testing MSE  122.244
     Training RMSE   12.333
      Testing RMSE   11.056
Training R-squared    0.963
 Testing R-squared    0.965

Conclusion: Underfitting: both errors are high
```




Comparison of Training and Testing Errors
Mean Squared Error
0
20
40
60
80
100
120
140
Training Error
Testing Error